

Fairness among TCP connections

Alaa Abo Mokh
Adan J bara
Adan Wattad



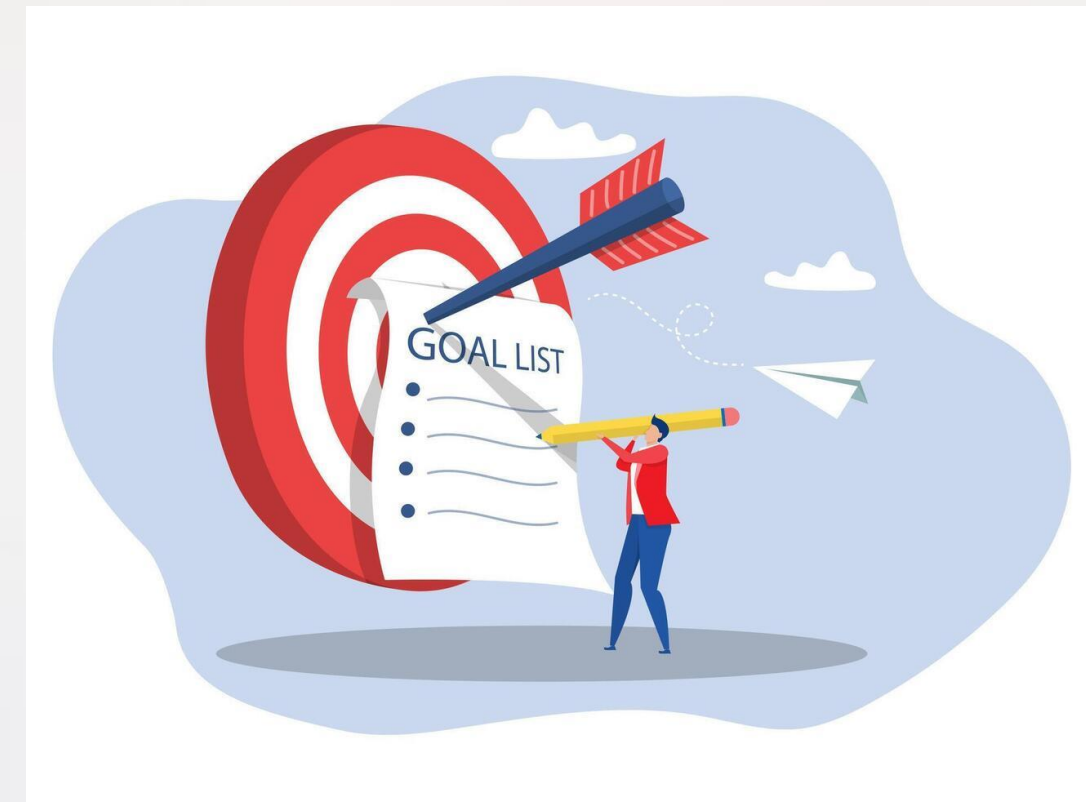
Project Goals

The objective of this project is to evaluate and compare the **fairness** of different congestion control protocols.

The evaluation is performed using a **Dumbbell topology** to simulate network conditions.

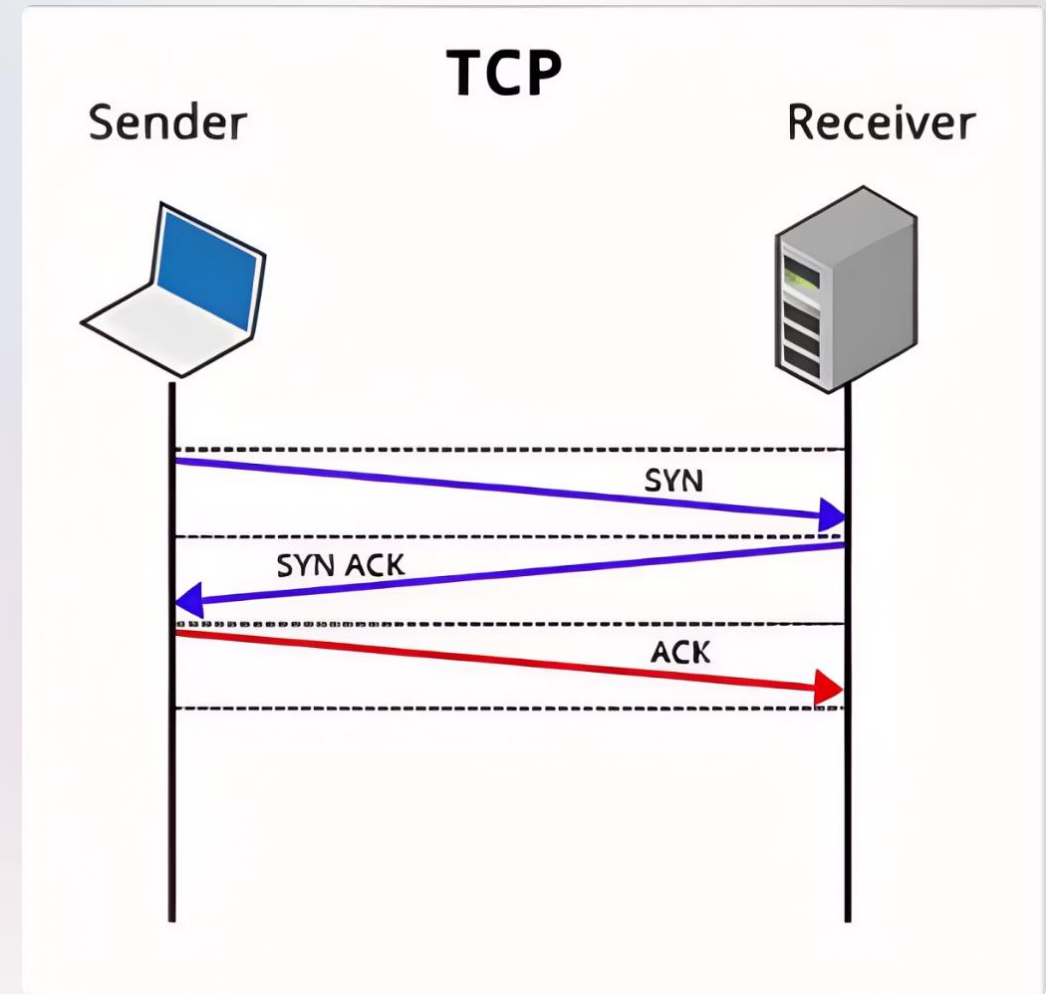
Fairness is assessed according to three key criteria:

- Jain's Index
- Max-Min Fairness



TCP

- Connection Oriented
- Congestion Control
- Reliable
- Flow Control
- Error Handling



TCP New Reno

Fast Recovery & Fast Retransmission

Retransmit lost packets on triple duplicate ACKs

AIMD

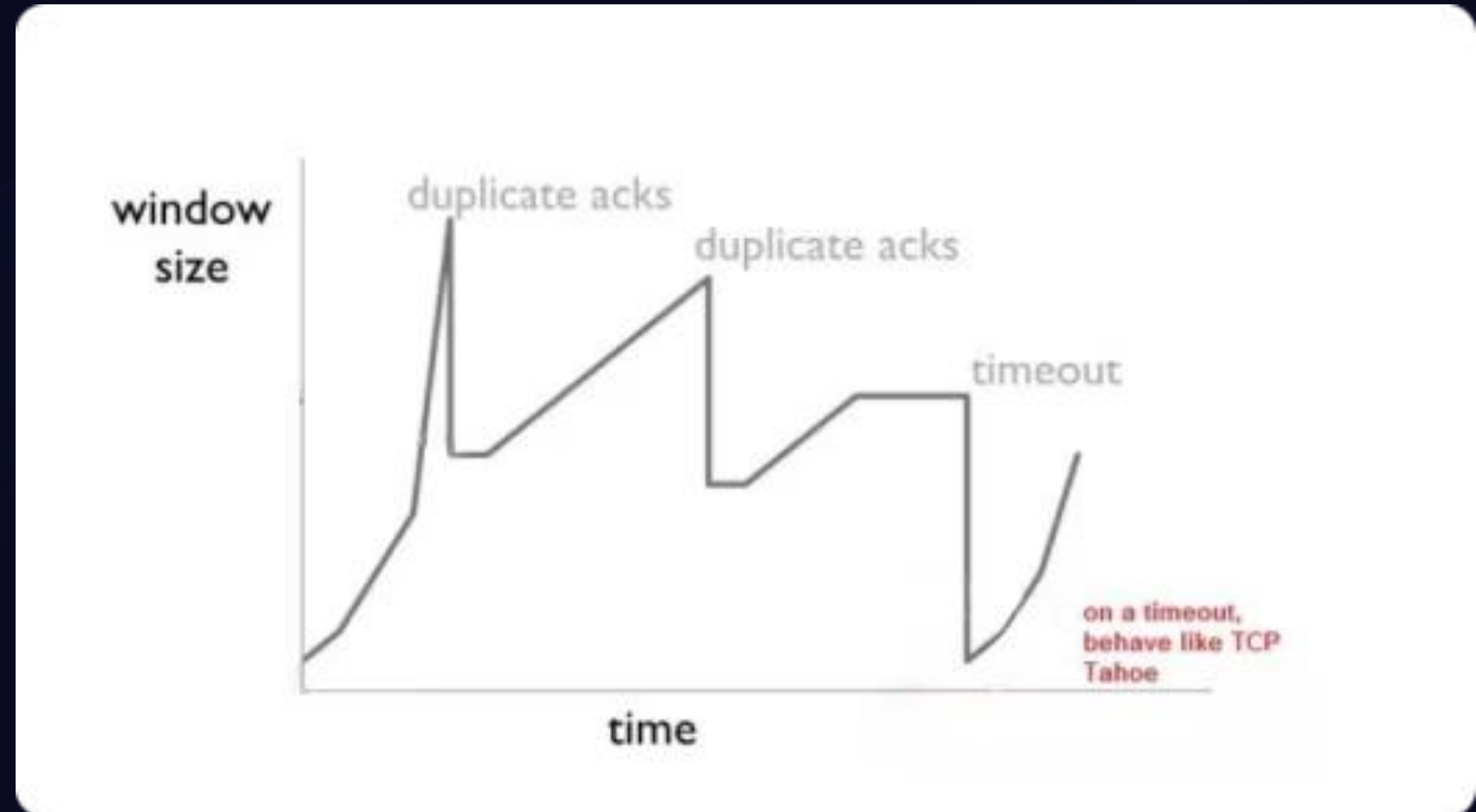
Adjusts sending rate dynamically

Supports Partial ACKs

Retransmit only missing packets

Better throughput and latency

Allows for continued transmission during fast recovery



BBR

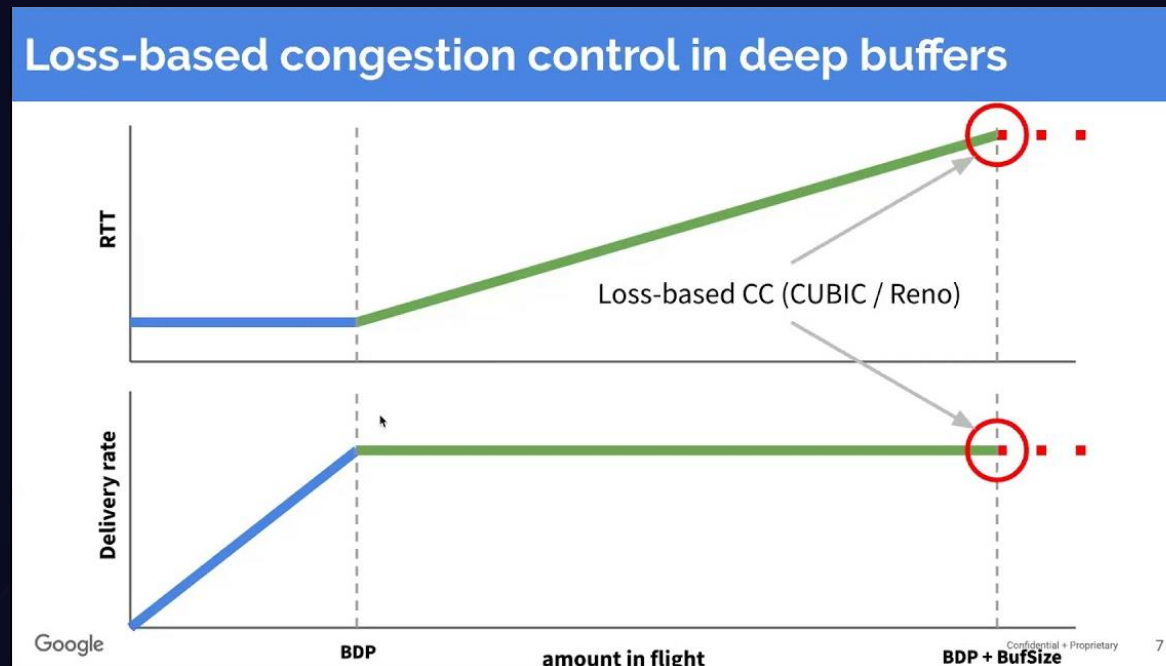
How BBR Works:

- Estimates Bottleneck Bandwidth
- Measures RTT
- Adjusts Congestion Window

Key Features of BBR:

- Optimized for high-bandwidth, high-latency
- Reduces packet loss
- Minimizes latency & Improves fairness

$$Cwnd = BtlBw \times Rtprop$$



CUBIC

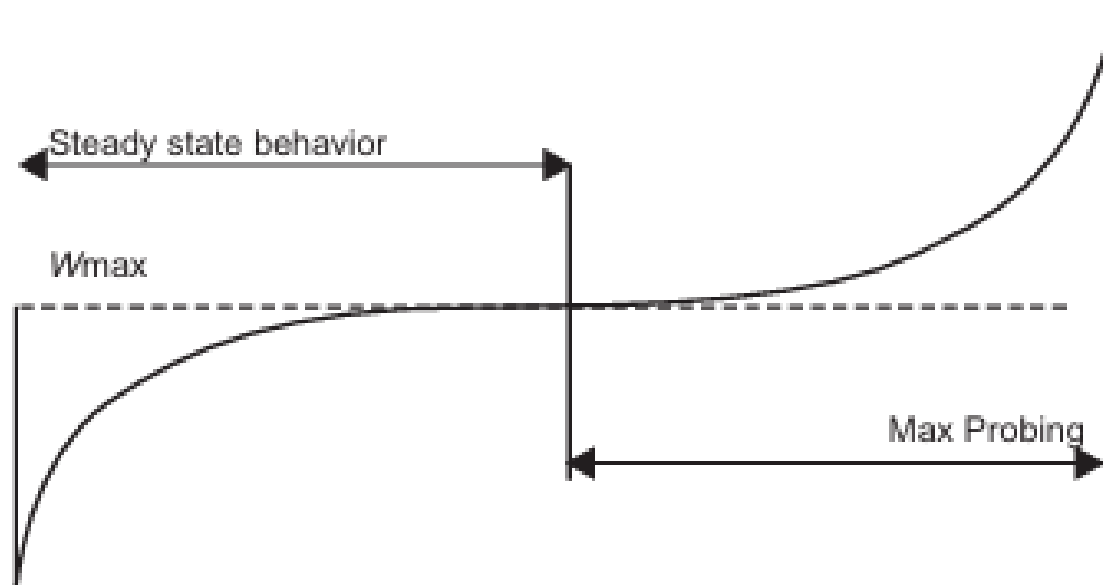
high-bandwidth, high-latency networks

Cubic window growth (not linear)

Detects congestion through packet loss

RTT-independent performance

In theory CUBIC is RTT-independent

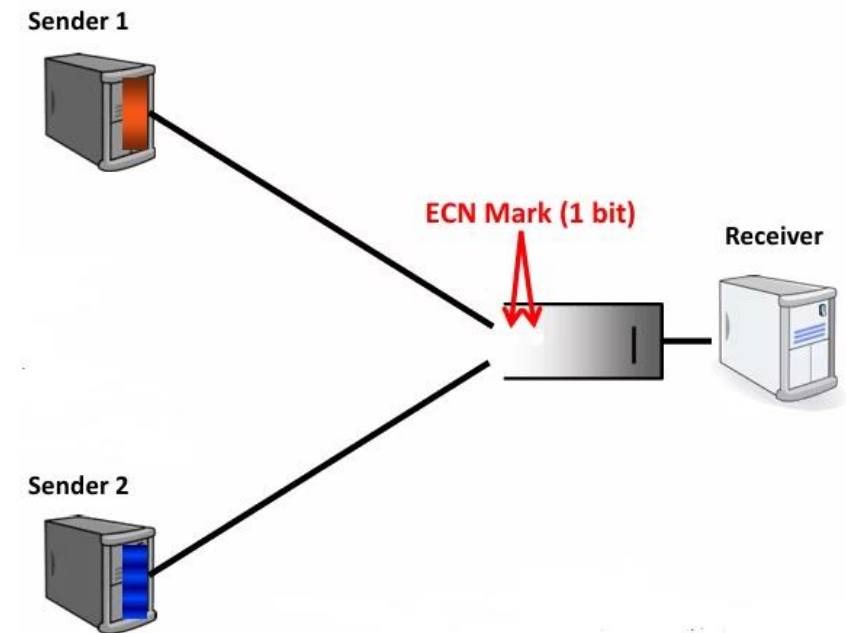


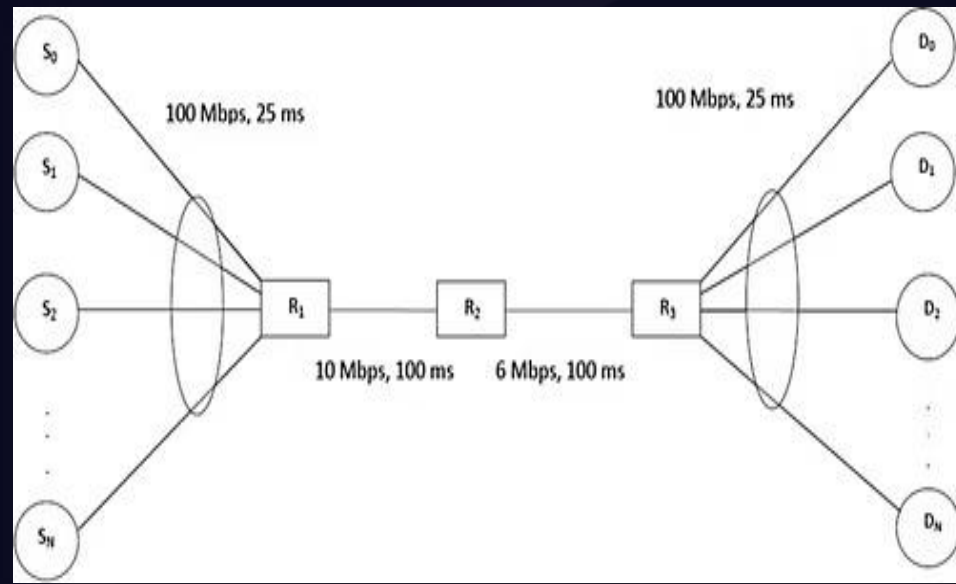
$$W(t) = C(t - K)^3 + W_{\max}$$

$$(\beta - 1) \cdot cwnd \leftarrow cwnd$$

DCTCP

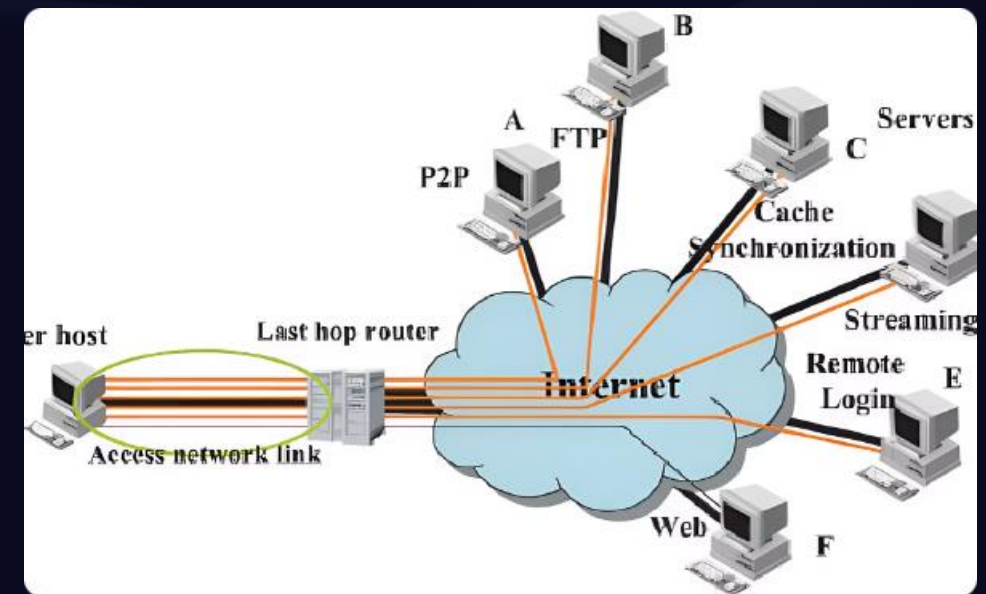
- Optimized for high-speed, low-latency **data centers**
- Uses **ECN** for early congestion detection
- Maintains **minimal packet loss**
- Achieves **high throughput**
- Ensures **low latency** communication





Dumbbell Topology-

- High-capacity center link
- Simulates bottlenecks



Bottleneck

- Bottlenecks occur in high-traffic zones
- Often found at the center link

Fairness Computation

Max-Min Fairness

One can not increase the rate of a flow, without decreasing the rate of another flow having equal or lesser rate

Jain's Index

measures the equality of resource distribution among users

$$\mathcal{J}(x_1, x_2, \dots, x_n) = \frac{(\sum_{i=1}^n x_i)^2}{n \cdot \sum_{i=1}^n x_i^2}$$





ns3

Purpose

a discrete-event network simulator, used for research and educational purposes.

Benefit

provides a realistic simulation environment for internet protocols

Key Feature

NS-3 supports modular design

Experiment Variations

- 1 Vary the RTT uniformly across simulations, keeping all flows within each simulation run assigned the *same* RTT value.
- 2 In each simulation, one flow has fixed RTT of 100 ms, while the other flow's RTT is scaled by a factor k to create RTT asymmetry.



Protocol Environment Types (What Protocols Are Used in Each Setup)

Environment Type 1 – Single Protocol

All senders use the **same TCP variant**, varied across runs: NewReno, BBR, CUBIC, DCTCP

Environment Type 2 – Two Protocols (Split Senders)

The sender population is split evenly between two protocols: NewReno & CUBIC, BBR & CUBIC, NewReno & BBR

- All simulations were conducted using two concurrent TCP flows to evaluate performance and fairness.

The project's stages

1. Learning stage

Studied the theoretical background and Learned how to use NS-3

2. Building the topology using NS-3

Dumbbell topology with 2–4 flows

3. Setting the bandwidth and other constants (e.g., delay) in the simulation

4. Defining the variables and their values

e.g. $rtt=0,20,40,60,100$ (Ms)

```
ns per param value
cs directory with timestamp
+ "%Y%m%d-%H%M%S")
results-$TIMESTAMP"
atch/$RESULTS_DIR"

E="scratch/$RESULTS_DIR/$5"
="scratch/$RESULTS_DIR/$6"

results file
TS_FILE"

in $4; do
"Testing $3=$val..."

R1="scratch/$RESULTS_DIR/tmp_thr1_${val}.txt"
R2="scratch/$RESULTS_DIR/tmp_thr2_${val}.txt"
THR1"
R2"

for this parameter value
N); do
3=$val"
```

5. Creating the script

```
wnd ב-MSS
ssBytes = 1448;

===== Helpers / Metrics =====
le JainIndex2(double x1, double x2)

le denom = 2.0 * (x1 * x1 + x2 * x2);
(denom <= 0.0) return 0.0;
double sum = x1 + x2;
return (sum * sum) / denom;

===== Tracers =====
tic void TraceThroughput(Ptr<FlowMonitor> monitor)

auto stats = monitor->GetFlowStats();
Time now = Now();

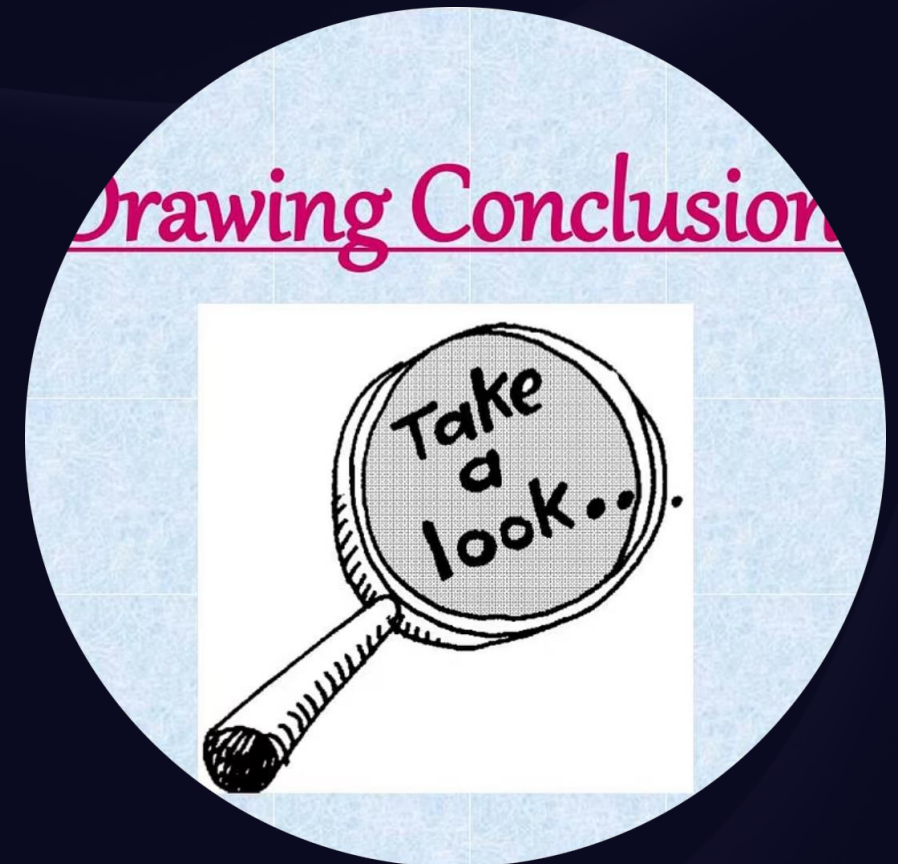
double r1 = -1.0; // Mbps (ערך שלילי = לא חושב בדגימה)
double r2 = -1.0;

const auto& kv : stats)

Classifier::FiveTuple t = classifier->Find

ress == Ipv4Address("10.0.2
```

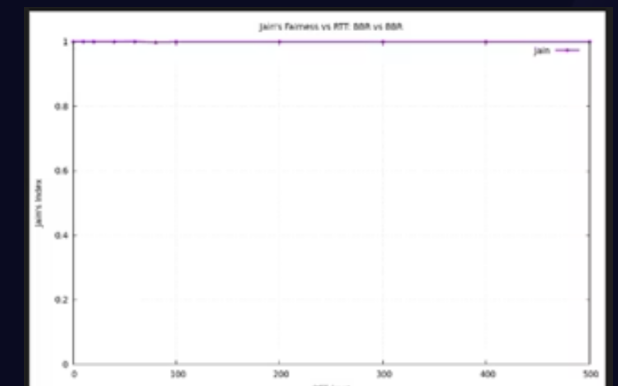
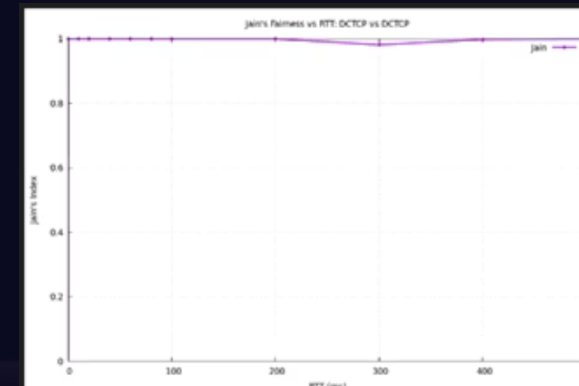
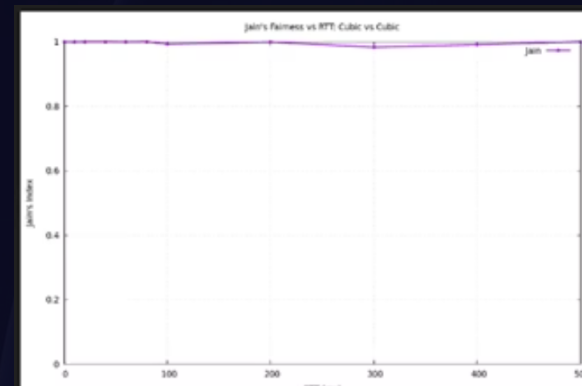
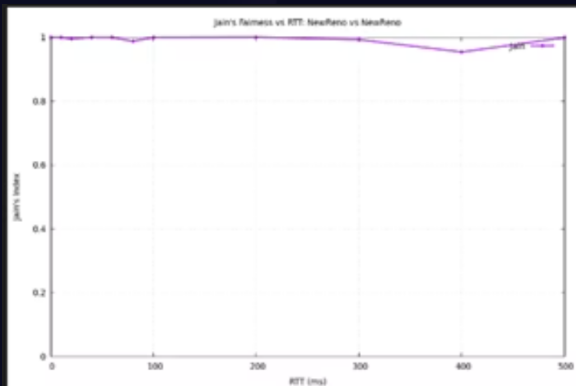
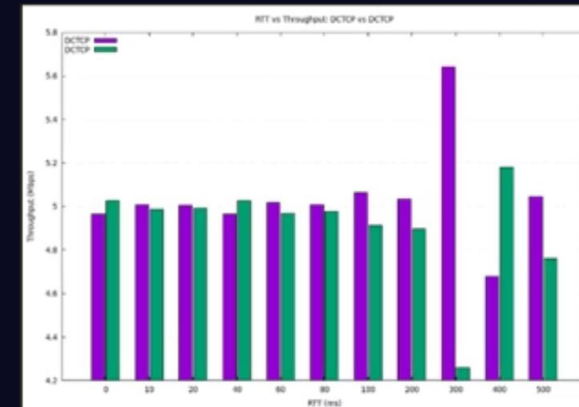
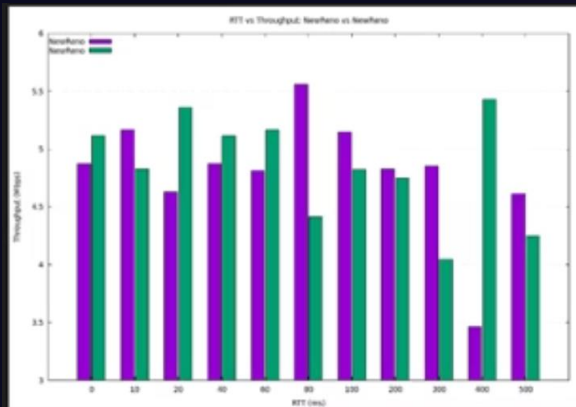
6. Running the simulations



7. Analyzing the results and drawing conclusions

Results- same RTT

Same Protocol



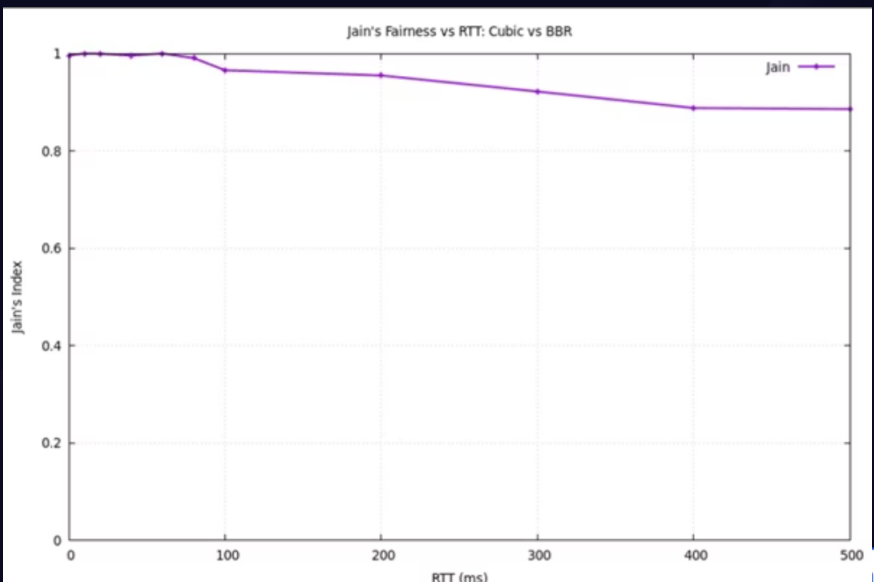
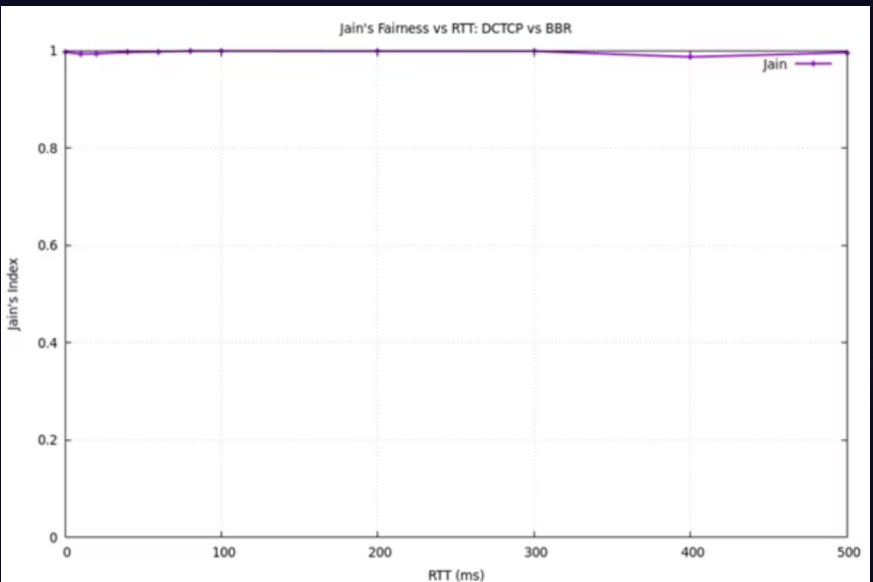
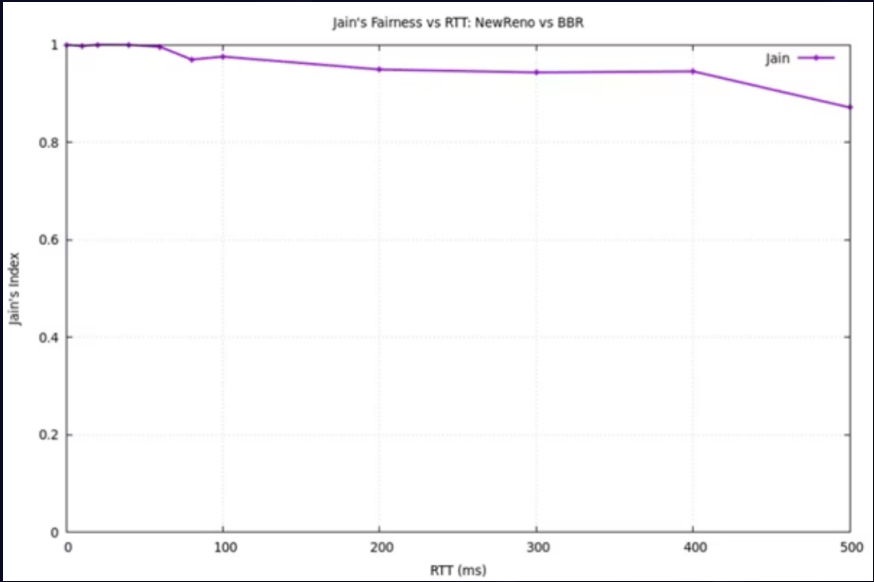
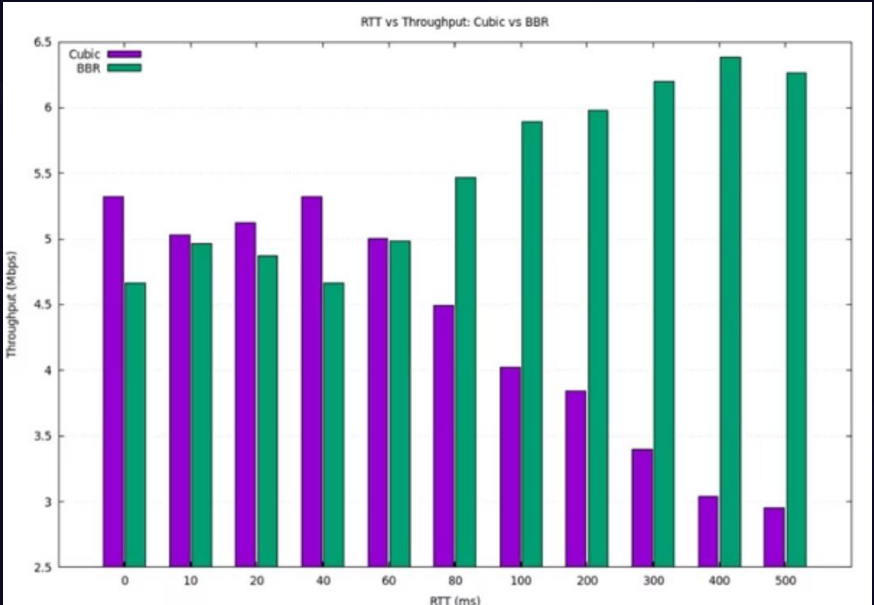
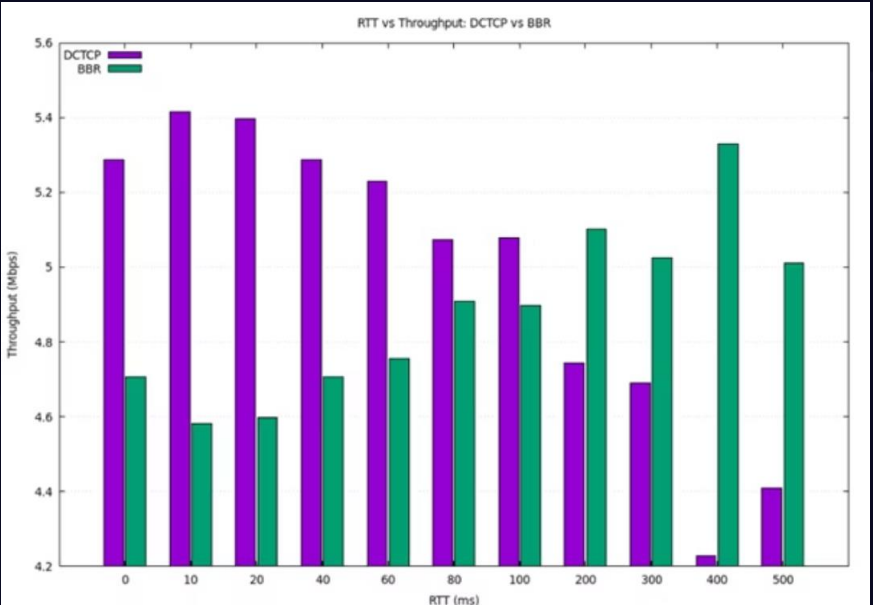
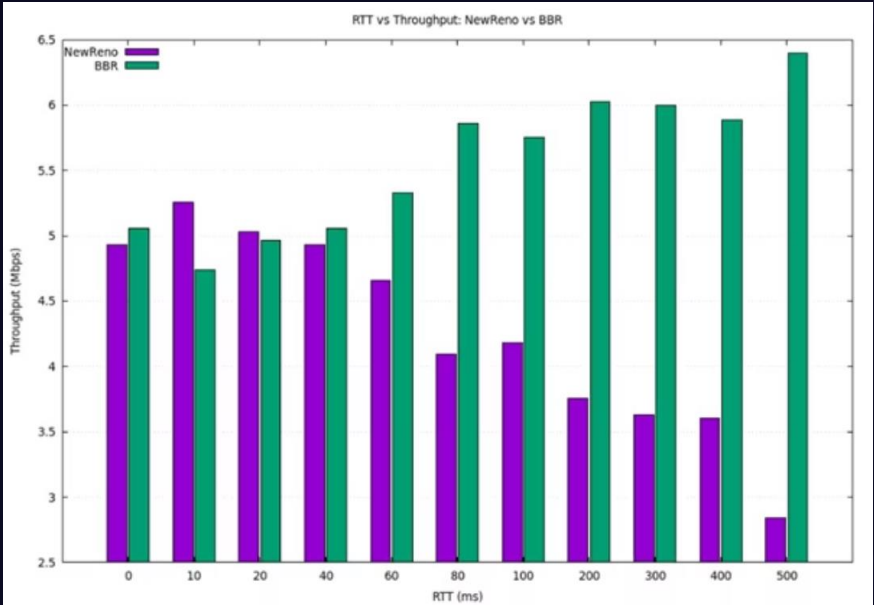
NewReno-NewReno

Cubic-Cubic

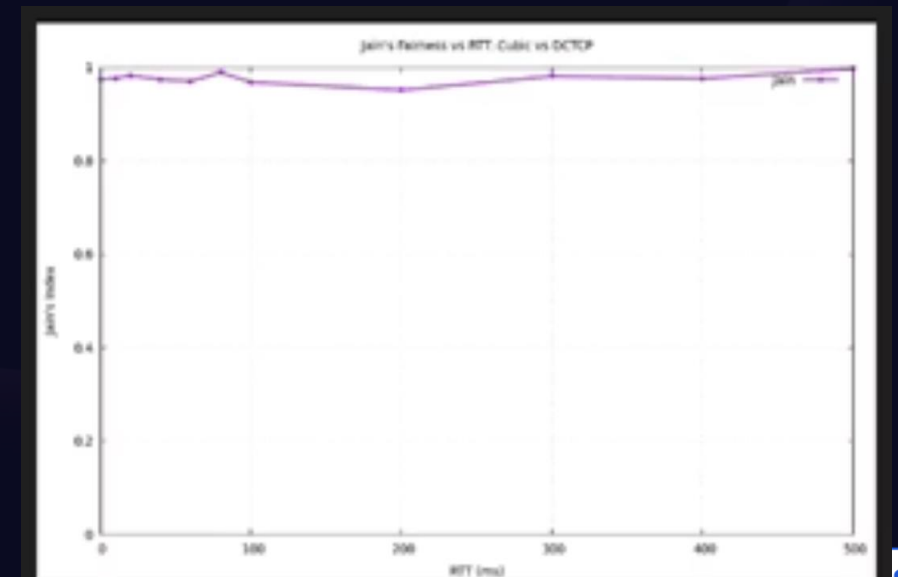
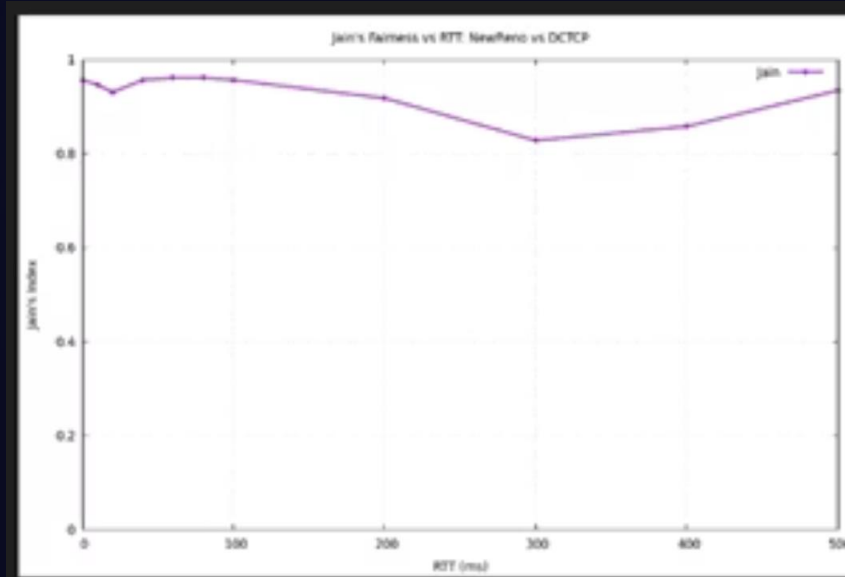
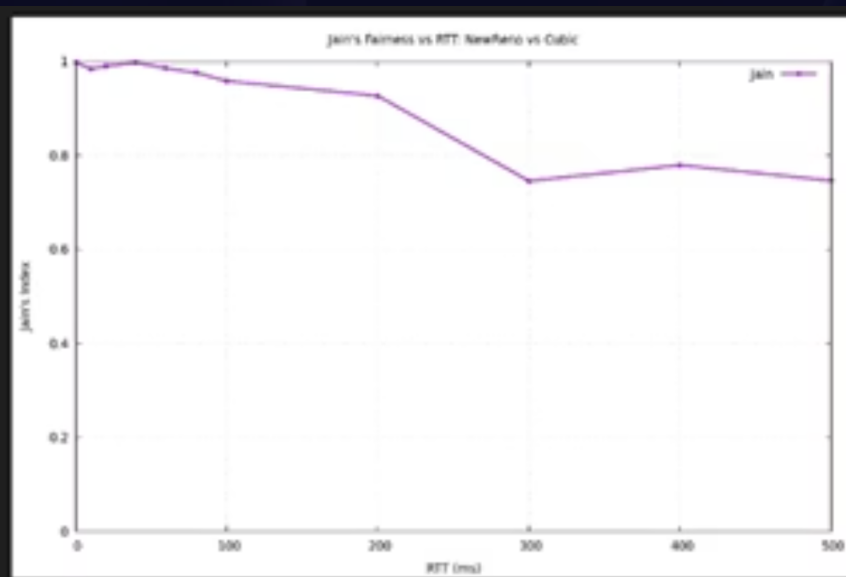
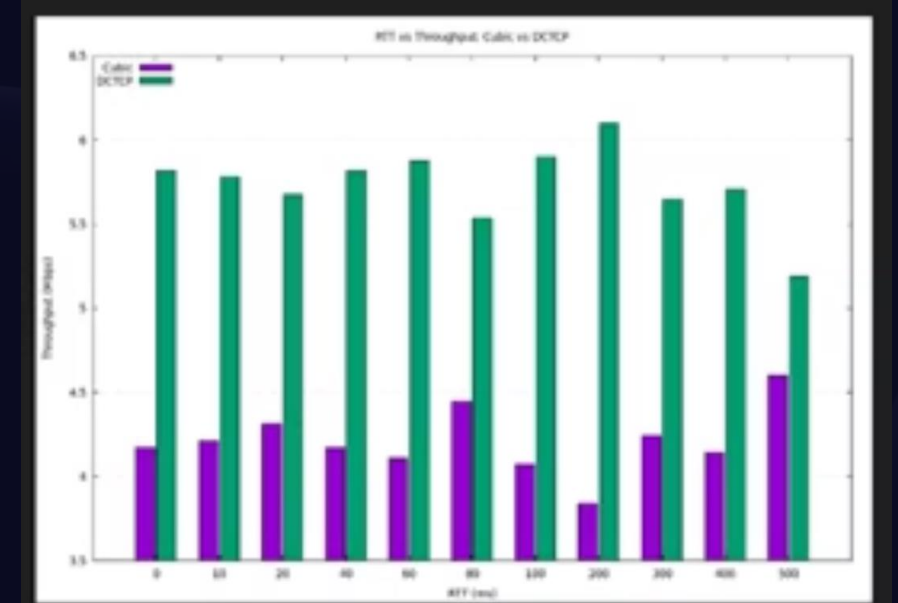
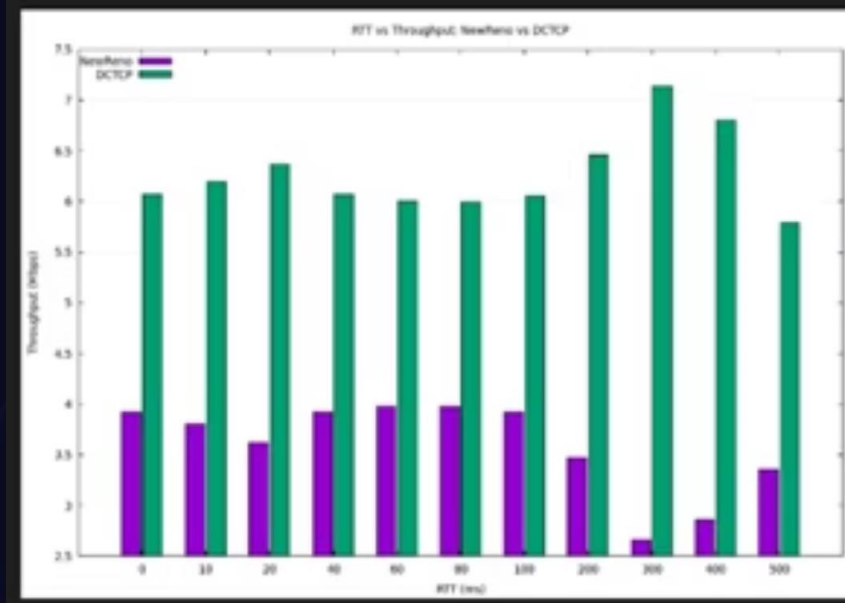
Dctcp-Dctcp

BBR-BBR

Results- same RTT different protocols

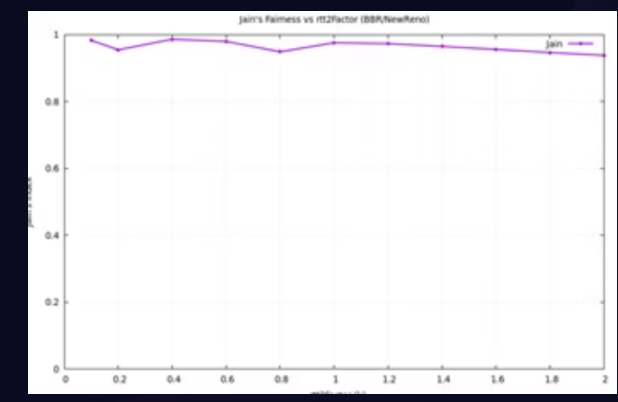
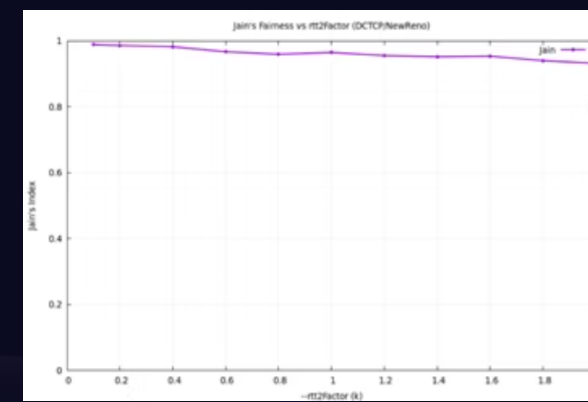
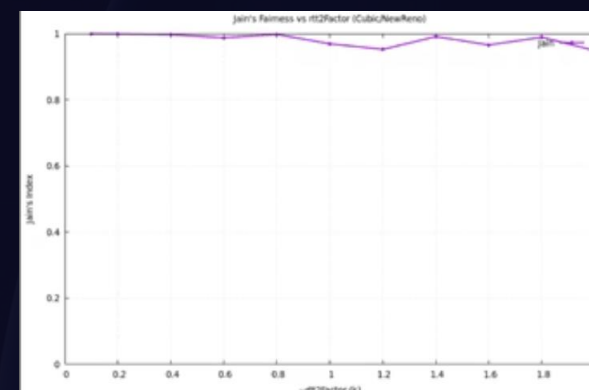
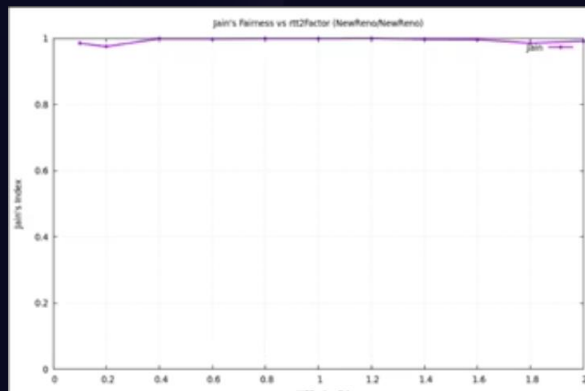
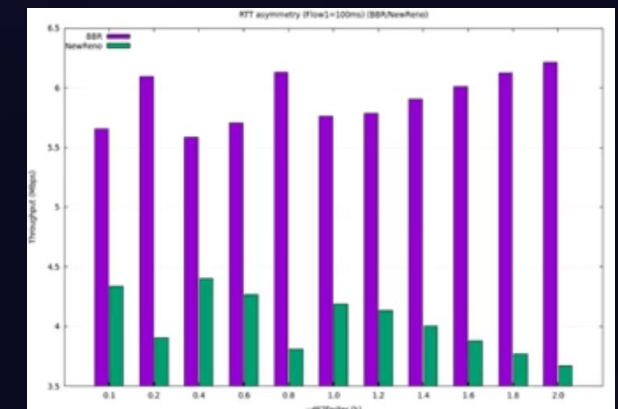
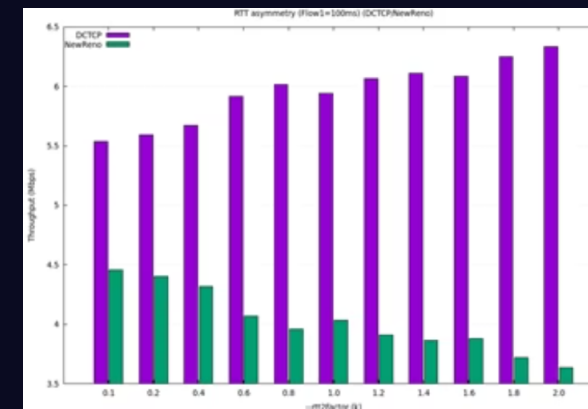
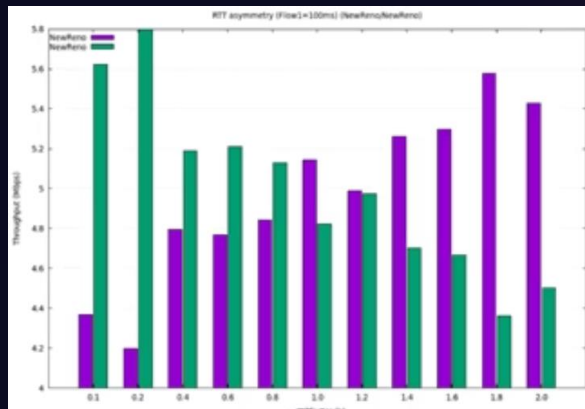


Results- same RTT different protocols



Results- different RTT

NewReno



NewReno-NewReno

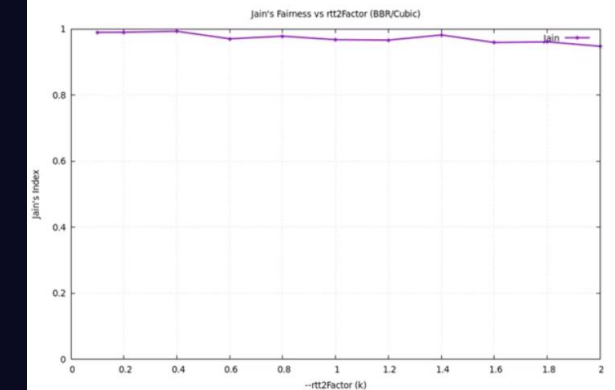
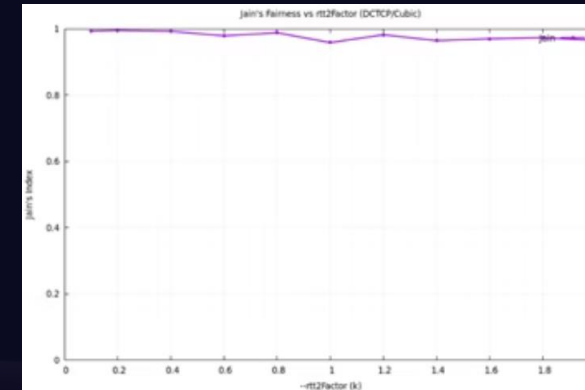
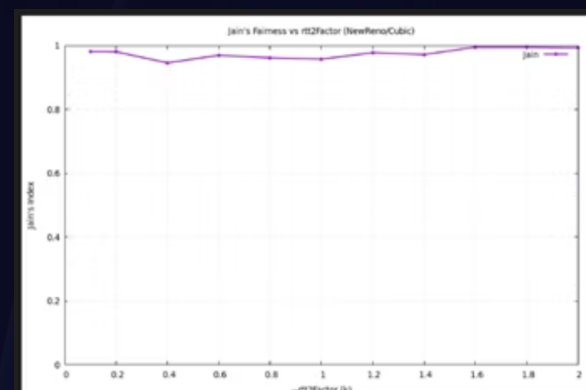
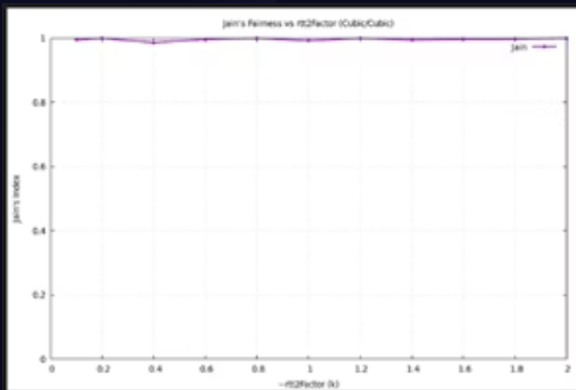
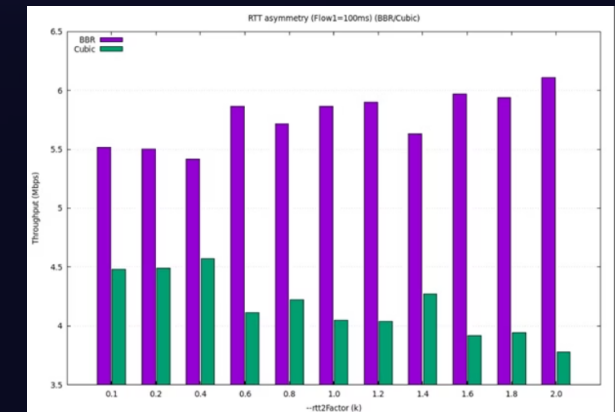
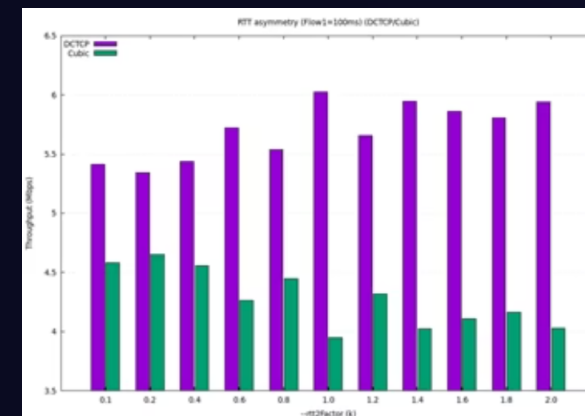
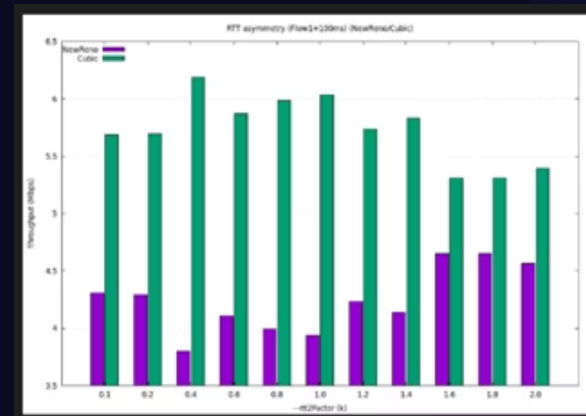
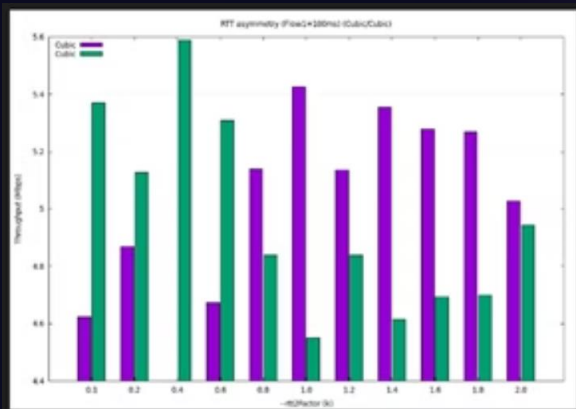
Cubic-NewReno

Dctcp-NewReno

BBR-NewReno

Results- different RTT

Cubic



Cubic-Cubic

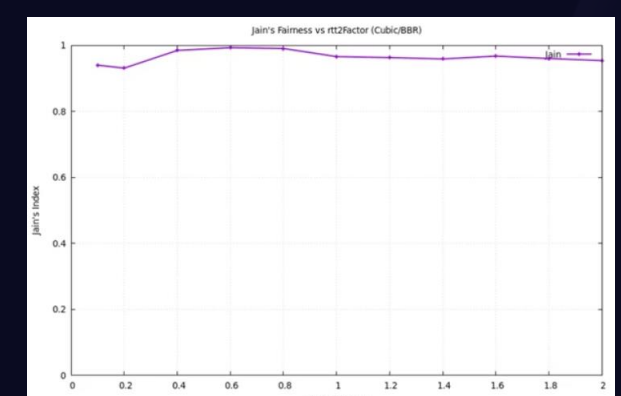
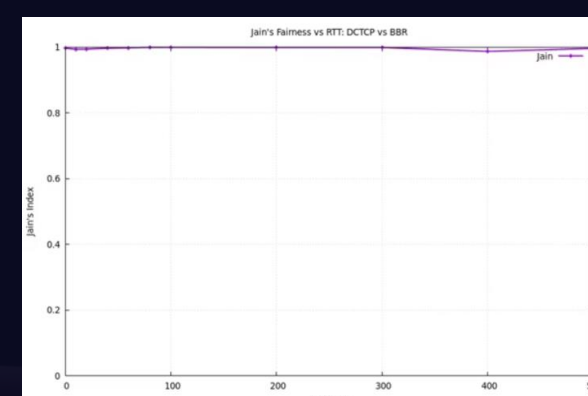
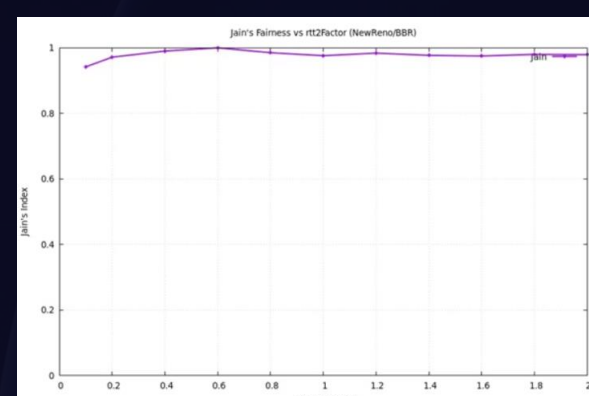
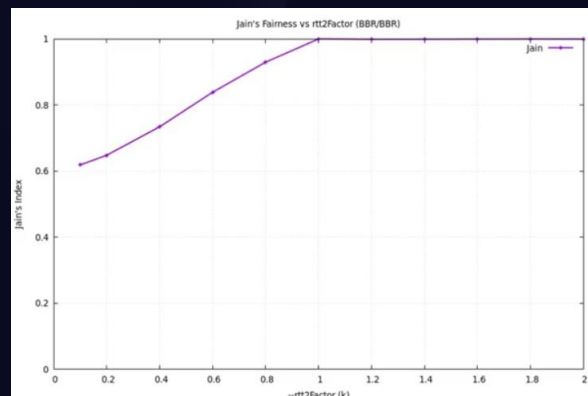
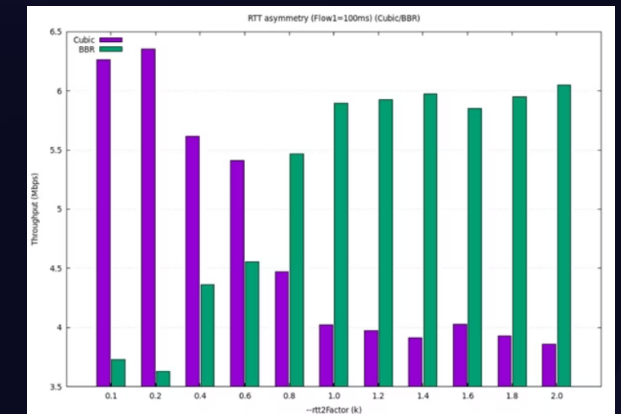
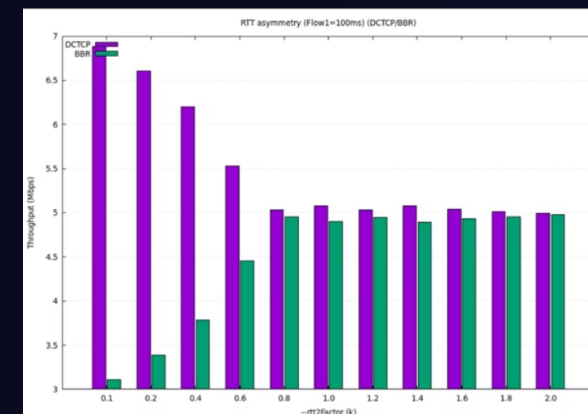
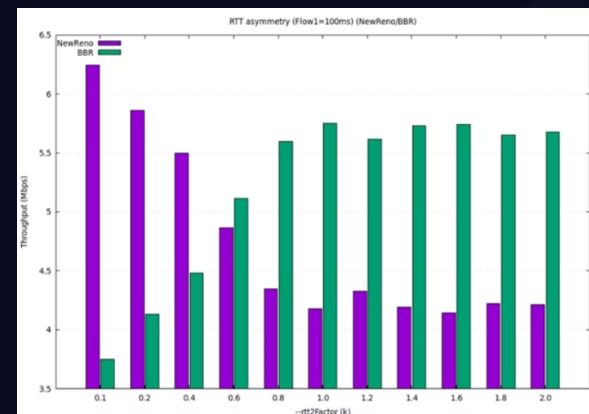
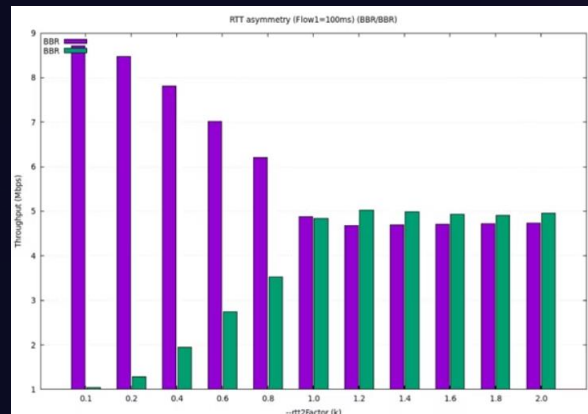
NewReno-Cubic

Dctcp-Cubic

BBR-Cubic

Results- different RTT

BBR



BBR-BBR

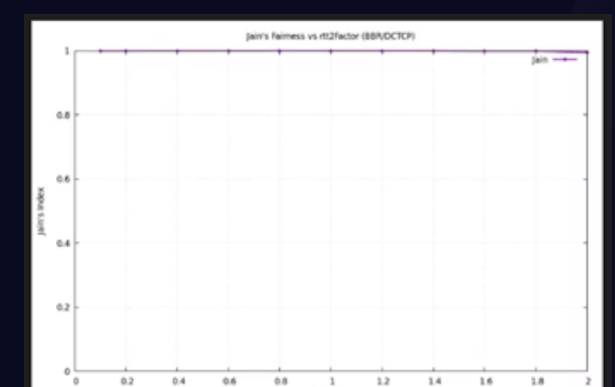
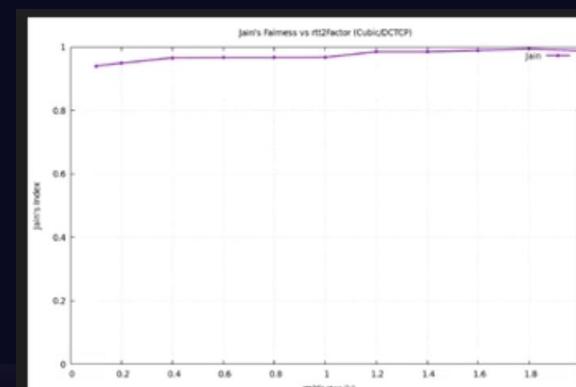
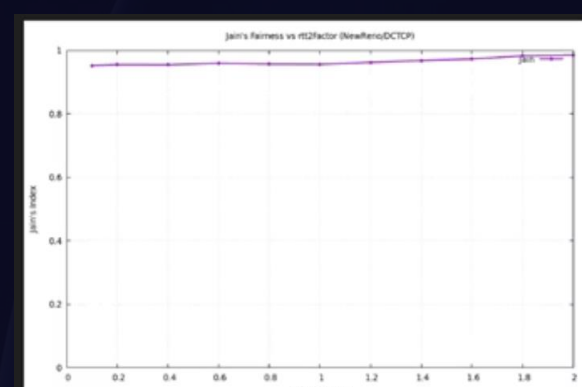
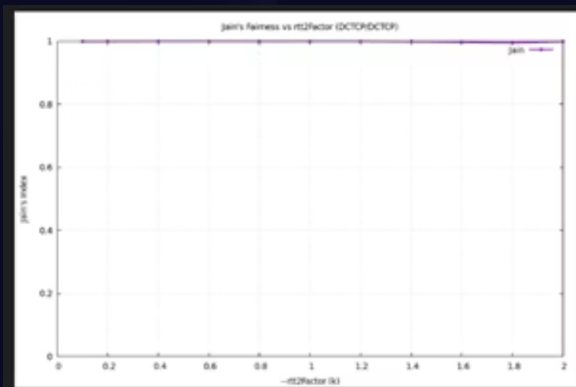
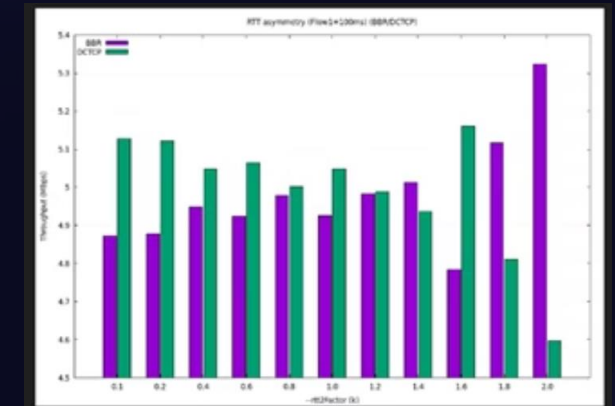
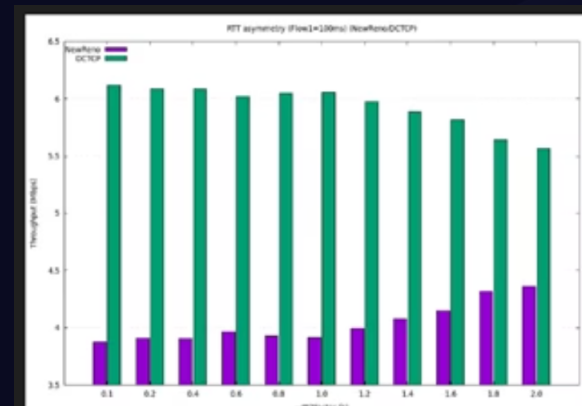
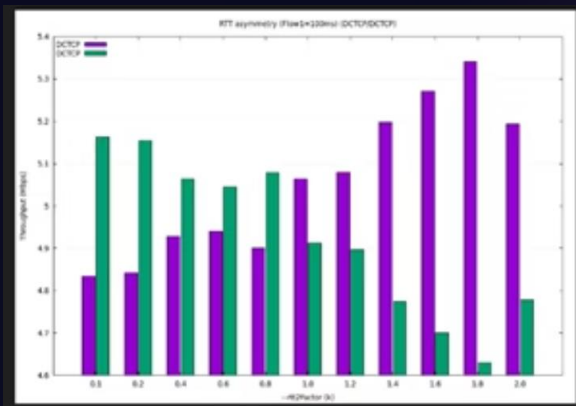
NewReno-BBR

Dctcp-BBR

Cubic-BBR

Results- different RTT

Dctcp



Dctcp-Dctcp

NewReno-Dctcp

Cubic-Dctcp

BBR-Dctcp

Conclusions

- When Dctcp is paired with loss based CCA's - NewReno, Cubic- it captures the bandwidth unrelated to RTT but due to its advanced algorithm dealing with congestion.
- NewReno gets much less throughput when paired with other CCA's so it nearly losses bandwidth every time.
- Cubic manages to get decent throughput when paired with other CCA's due to its fast growing cwnd.
- when BBR has similar RTT to Dctcp and itself it's protocol friendly. It manages to take appropriate throughput without outperforming the other protocol. But' with NewReno and Cubic it causes outperformances.
- The Max-Min Fairness principle holds when both protocols are the same and both flows have the same RTT. In other cases, it is difficult to determine, since both flows attempt to fully utilize the available bandwidth, but in practice they do not receive an equal share of it.
- Modern protocols such as BBR and DCTCP are capable of sharing bandwidth efficiently and fairly, even under complex network conditions.

References & Appendix

GitHub Repository:

<https://github.com/alaajalalabomokh/Fairness-among-TCP-connections.git>

References :

- [What is TCP \(Transmission Control Protocol\)? - GeeksforGeeks](#)
- [Wayback Machine](#)
- [RFC 6582: The NewReno Modification to TCP's Fast Recovery Algorithm](#)
- [DCTCP \(DataCenter TCP\) — The Linux Kernel documentation](#)
- [TCP Fairness Among Modern TCP Congestion Control Algorithms Including TCP BBR | IEEE Conference Publication | IEEE Xplore](#)